Chapter 7 CRT

1. Which members of the Circle class are encapsulated?

Encapsulation means keeping data hidden (private) and controlling access through methods.

Private variables (like `radius`) are encapsulated.

2. What name must the constructor of a class have?

The constructor must have the exact same name as the class.

Example:

```
public class Circle {
    public Circle() {
        // constructor
    }
}
```

3. Explain the difference between the private and public access modifiers.

Private

- Can only be accessed inside the same class
- Used to protect data (encapsulation)

Public

- Can be accessed from anywhere (other classes too)

private = hidden

public = open to everyone

4. Consider the following code. Is the last statement valid or invalid? Explain.

```
Circle dot = new Circle(2);
dot.radius = 5;
```

Invalid - It is invalid because radius is usually declared private. You cannot access private variables directly from outside the cl.

5. Use the following class to answer the questions below:

```
public class Roo {
private int x;
public Roo f
X= 1
public void setX(int z) (
X = 2,
public int getX) {
return (x);
public int calculatel (
x = x * factor;
return (x);
private int factor ( return (0.12);
```

a) What is the name of the class?

Roo

b) What is the name of the data member?

x

c) List the accessor method.
Accessor = gets value
`getX()`

d) List the modifier method.
Modifier = changes value
`setX(int z)`

e) List the helper method.
Helper = used internally
`factor()`

f) What is the name of the constructor?
`Roo`
(Constructors always match the class name)

g) How many method members are there?
5 methods

6. What is the difference between a class and an object?

Class

- A blueprint/template
- Defines variables and methods

Example: `Circle`

Object

- A real instance of a class

Example: `Circle c1 = new Circle();`

9. Use the following class data member definitions to answer the questions below:

```
public class Moo {
private double y;
private static int x;
private static final z;
```

a) Which data member is a constant?

z

- `static final` means it is a constant
- Its value cannot change after it is set

b) Which data members are variables?

y and x

- y = instance variable (changes per object)
- x = class variable (shared, but still a variable)

c) Which data member(s) are instance members?

y

- Instance members belong to each object individually
- Each object of Moo gets its own copy of y

d) Which data member(s) are class members?

x and z

- `static` means it belongs to the class itself, not objects
- x = class variable
- z = class constant

11. Compare and contrast overriding methods to overloading methods.

Overloading

- Same method name, but different inputs (type, number, or order)
- Happens within the same class
- Used to make methods more flexible

Example: `void add(int a, int b)`

```
void add(double a, double b)
```

Overriding

- Same method name and same inputs
- Happens in different classes (inheritance)
- The child class replaces the parent class method

Example:
```
class Animal {

    void sound() { }

}

class Dog extends Animal {

    void sound() { } // overrides parent method

}
```